SERVICIO NACIONAL DE APRENDIZAJE - SENA

Centro de Formación: SENA - Cartagena, Bolívar

SISTEMA DE GESTIÓN DE USUARIOS

(SGU)

Documentación Técnica del Proyecto

Evidencia: GA7-220501096-AA3-EV01
Nombre de la Evidencia: Codificación de módulos del software stand-alone, web y móvil

Programa: Análisis y Desarrollo de Software (ADSO)
Aprendiz: Juan Andrés Pérez Elguedo
Instructor: Fireman

Año: 2026

TABLA DE CONTENIDO

1. Introducción
2. Objetivos
3. Justificación
4. Descripción del Proyecto
5. Stack Tecnológico
6. Arquitectura del Sistema
7. Estructura del Proyecto
8. Base de Datos
9. API REST (Endpoints)
10. Manual de Instalación
11. Manual de Uso
12. Guía de Pruebas
13. Solución de Problemas
14. Conclusiones
15. Bibliografía

1. INTRODUCCIÓN

El presente documento describe la implementación técnica del **Sistema de Gestión de Usuarios (SGU)**, un software web completo desarrollado como parte de la evidencia **GA7-220501096-AA3-EV01** del programa de formación **Análisis y Desarrollo de Software (ADSO)** del **Servicio Nacional de Aprendizaje (SENA)**.

El proyecto nace de la necesidad de contar con una herramienta moderna, segura y eficiente para la administración de usuarios dentro de una organización. El sistema permite:

- Registrar, editar y eliminar usuarios de forma controlada
- Autenticar usuarios mediante tokens JWT
- Gestionar roles y permisos diferenciados
- Auditar todas las acciones realizadas en el sistema

Para su desarrollo se aplicaron conceptos fundamentales de ingeniería de software: análisis de requerimientos, diseño de arquitectura en capas, modelado de bases de datos normalizadas, uso de frameworks modernos y buenas prácticas de codificación.

El documento se estructura en secciones que abarcan desde los objetivos del proyecto hasta los manuales de instalación y uso, sirviendo como guía completa para cualquier persona que necesite entender, mantener o desplegar el sistema.

2. OBJETIVOS

2.1 Objetivo General

Desarrollar un sistema web completo para la gestión de usuarios que integre backend, frontend y base de datos, aplicando los conocimientos adquiridos durante el programa de formación ADSO del SENA.

2.2 Objetivos Específicos

- Implementar un backend REST en Python con Flask que exponga todos los endpoints necesarios para la gestión de usuarios.
- Desarrollar un frontend en Vue.js 3 que consuma la API y proporcione una interfaz intuitiva y responsive.
- Diseñar una base de datos relacional en MySQL normalizada hasta la Tercera Forma Normal (3FN).
- Implementar autenticación segura mediante JWT y cifrado de contraseñas con bcrypt.
- Aplicar el principio de eliminación lógica para garantizar la integridad de los datos.
- Registrar todas las acciones importantes en una tabla de auditoría.
- Documentar el sistema de forma clara para facilitar su mantenimiento y despliegue.

3. JUSTIFICACIÓN

El desarrollo de este proyecto responde a varias necesidades:

Académicas: Permite al aprendiz aplicar de forma integral los conocimientos adquiridos durante el programa ADSO, incluyendo análisis, diseño, codificación, pruebas y documentación.

Técnicas: Implementa tecnologías modernas y demandadas en el mercado laboral (Python, Flask, Vue.js, MySQL, JWT), demostrando competencia en el desarrollo full-stack.

Prácticas: Ofrece una solución real a un problema común en cualquier organización: la gestión segura y controlada de usuarios.

De seguridad: Refleja la importancia de aplicar principios como el cifrado de contraseñas, la autenticación por tokens y la auditoría de acciones, aspectos críticos en el desarrollo de software profesional.

4. DESCRIPCIÓN DEL PROYECTO

4.1 ¿Qué es el SGU?

El Sistema de Gestión de Usuarios (SGU) es una aplicación web que permite a las organizaciones administrar de forma centralizada a los usuarios que tienen acceso a sus sistemas informáticos.

4.2 Funcionalidades Principales

RF	Funcionalidad	Descripción
RF-01	Registro de usuarios	Los usuarios pueden crear una cuenta con nombre, correo único y contraseña segura
RF-02	Edición de usuarios	Los administradores pueden modificar los datos de cualquier usuario
RF-03	Eliminación lógica	Los usuarios se desactivan (no se borran) para preservar el historial
RF-04	Inicio de sesión	Autenticación con JWT para acceso seguro
RF-05	Gestión de roles	Asignación de roles: Administrador, Editor, Lector
RF-06	Auditoría	Registro automático de todas las acciones importantes

4.3 Requerimientos No Funcionales

RNF	Requerimiento	Implementación
RNF-01	Seguridad	JWT + bcrypt + HTTPS (futuro)
RNF-02	Rendimiento	Respuestas < 3 segundos
RNF-03	Base de datos	MySQL relacional normalizada 3FN
RNF-04	Protección de datos	Políticas de privacidad y control de acceso

4.4 Usuarios del Sistema

- **Administrador:** Control total del sistema
- **Editor:** Puede editar usuarios pero no eliminarlos
- **Lector:** Solo puede visualizar información

5. STACK TECNOLÓGICO

5.1 Backend

Tecnología	Versión	Uso
Python	3.14.8	Lenguaje principal
Flask	3.1.3	Framework web
SQLAlchemy	2.1.1	ORM
PyMySQL	1.2.3	Driver MySQL
Flask-JWT-Extended	4.7.4	Autenticación JWT
bcrypt	5.0.0	Cifrado de contraseñas
Flask-CORS	6.0.5	Manejo de CORS

5.2 Frontend

Tecnología	Versión	Uso
Vue.js	3.5.42	Framework progresivo
Vite	8.3.0	Bundler y servidor
Vue Router	4.6.4	Enrutamiento
Axios	1.20.0	Cliente HTTP

5.3 Base de Datos

Tecnología	Versión	Uso
MySQL	8.0+	Base de datos relacional
MySQL Workbench	Última	Gestor visual

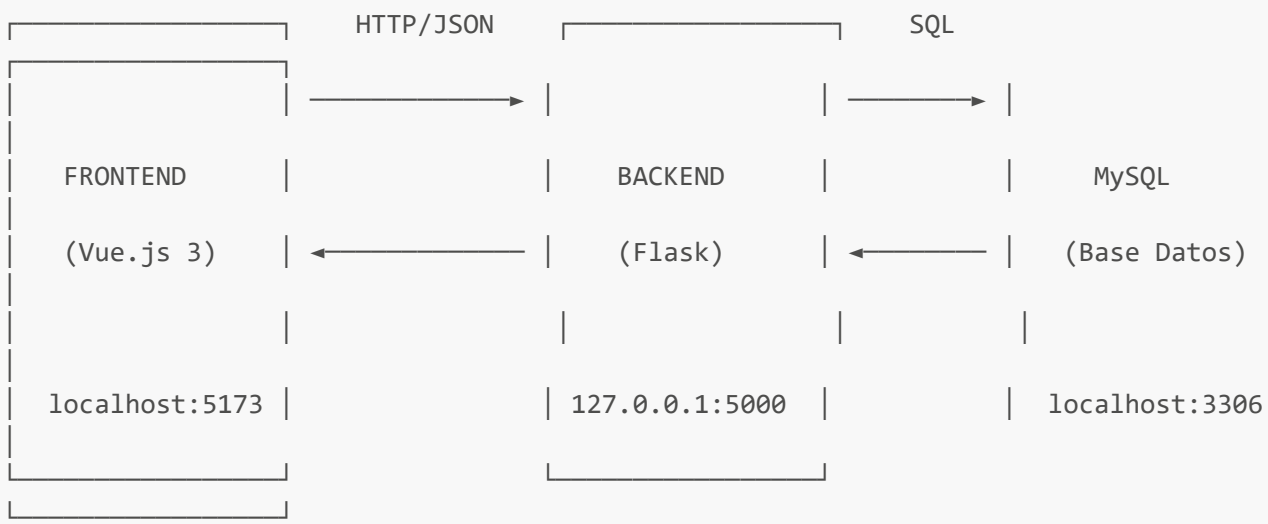
5.4 Herramientas de Desarrollo

- **VS Code** — Editor de código
- **Git** — Control de versiones
- **GitHub** — Repositorio remoto
- **Thunder Client** — Pruebas de API

6. ARQUITECTURA DEL SISTEMA

6.1 Arquitectura General

El sistema sigue una arquitectura **cliente-servidor de 3 capas**:



6.2 Capas del Backend

El backend está organizado en capas para separar responsabilidades:

Capa	Carpeta	Responsabilidad
Modelos	`models/`	Representan las tablas de la BD
Rutas	`routes/`	Definen los endpoints HTTP
Servicios	`services/`	Contienen la lógica de negocio
Utilidades	`utils/`	Funciones auxiliares (bcrypt)
Configuración	`config.py`	Configuración general
Extensiones	`extensions.py`	SQLAlchemy y JWT

6.3 Patrón de Diseño del Frontend

El frontend aplica el **patrón MVC (Modelo-Vista-Controlador)** de forma implícita:

- **Modelo:** Los datos vienen del backend vía API REST
- **Vista:** Los componentes `.vue` renderizan la interfaz
- **Controlador:** La lógica en `<script setup>` maneja eventos y llamadas a la API

7. ESTRUCTURA DEL PROYECTO

```
sgu/
│
├── sgu-backend/                    # BACKEND (Python + Flask)
│   ├── models/
│   │   ├── init.py
│   │   ├── usuario.py
│   │   ├── rol.py
│   │   ├── permiso.py
│   │   └── auditoria.py
│   ├── routes/
│   │   ├── auth.py
│   │   ├── usuarios.py
│   │   ├── roles.py
│   │   └── auditoria.py
│   ├── services/
│   │   ├── auth_service.py
│   │   ├── usuario_service.py
│   │   ├── rol_service.py
│   │   └── auditoria_service.py
│   ├── utils/
│   │   └── security.py
│   ├── venv/                       # Entorno virtual
│   ├── .env                        # Variables de entorno
│   ├── app.py                      # Punto de entrada
│   ├── config.py
│   ├── extensions.py
│   ├── database.sql                # Script de BD
│   └── requirements.txt
│
├── sgu-frontend/                   # FRONTEND (Vue.js 3)
│   ├── src/
│   │   ├── assets/styles/
│   │   │   └── global.css
│   │   ├── components/layout/
│   │   │   ├── Navbar.vue
│   │   │   ├── Footer.vue
│   │   │   └── Sidebar.vue
│   │   ├── views/
│   │   │   ├── auth/               # 6 pantallas de login/registro
│   │   │   ├── dashboard/          # 4 pantallas del panel
│   │   │   ├── logout/             # 3 pantallas de logout
│   │   │   └── general/            # 2 pantallas generales
│   │   ├── router/index.js
│   │   ├── services/api.js
│   │   ├── App.vue
│   │   └── main.js
│   ├── index.html
```

```
│   ├── package.json
│   └── vite.config.js
│
├── sgu-frontend_original/         # Respaldo HTML original
│
└── docs/                          # Documentación
```

8. BASE DE DATOS

8.1 Nombre y Configuración

- **Nombre:** `gestion_usuarios_db`
- **Motor:** InnoDB
- **Charset:** utf8mb4
- **Normalización:** Tercera Forma Normal (3FN)

8.2 Tablas

Tabla: `usuario`

Campo	Tipo	Descripción
id_usuario	INT PK AUTO	ID único
nombre	VARCHAR(100)	Nombre completo
correo	VARCHAR(150) UNIQUE	Correo único
contrasena	VARCHAR(255)	Hash bcrypt
estado	BOOLEAN	Activo/Inactivo
fecha_creacion	TIMESTAMP	Fecha de registro
ultimo_acceso	TIMESTAMP	Último login

Tabla: `rol`

Campo	Tipo	Descripción
id_rol	INT PK AUTO	ID único
nombre_rol	VARCHAR(100)	Administrador/Editor/Lector
descripcion	VARCHAR(255)	Descripción

Tabla: `permiso`

Campo	Tipo	Descripción
id_permiso	INT PK AUTO	ID único
nombre_permiso	VARCHAR(100)	Nombre del permiso
descripcion	VARCHAR(255)	Descripción

Tabla: `usuario_rol` (N:M)

Campo	Tipo	Descripción
id_usuario	INT FK	Referencia a usuario
id_rol	INT FK	Referencia a rol

Tabla: `rol_permiso` (N:M)

Campo	Tipo	Descripción
id_rol	INT FK	Referencia a rol
id_permiso	INT FK	Referencia a permiso

Tabla: `auditoria`

Campo	Tipo	Descripción
id_auditoria	INT PK AUTO	ID único
id_usuario	INT FK	Usuario que hizo la acción
accion	VARCHAR(255)	Descripción de la acción
fecha	TIMESTAMP	Fecha y hora
ip	VARCHAR(45)	IP del cliente

8.3 Datos Iniciales

- 3 roles: Administrador, Editor, Lector
- 7 permisos del sistema
- 1 usuario admin inicial

9. API REST (ENDPOINTS)

9.1 Autenticación (público)

Método	Endpoint	Descripción
POST	`/api/auth/registro`	Registrar nuevo usuario
POST	`/api/auth/login`	Iniciar sesión, devuelve JWT

9.2 Usuarios (requiere JWT)

Método	Endpoint	Descripción
GET	`/api/usuarios`	Listar usuarios (con filtros)
GET	`/api/usuarios/<id>`	Ver un usuario
PUT	`/api/usuarios/<id>`	Editar usuario
DELETE	`/api/usuarios/<id>`	Eliminación lógica
PUT	`/api/usuarios/<id>/reactivar`	Reactivar usuario

9.3 Roles y Permisos (requiere JWT)

Método	Endpoint	Descripción
GET	`/api/roles`	Listar roles
GET	`/api/roles/<id>`	Ver un rol
POST	`/api/roles`	Crear rol
POST	`/api/roles/asignar`	Asignar rol a usuario
GET	`/api/permisos`	Listar permisos

9.4 Auditoría (requiere JWT)

Método	Endpoint	Descripción
GET	`/api/auditoria`	Listar registros
GET	`/api/auditoria/usuario/<id>`	Registros de un usuario
GET	`/api/auditoria/resumen`	Total de registros

9.5 Utilidades

Método	Endpoint	Descripción
GET	`/api/health`	Verificar estado del backend

10. MANUAL DE INSTALACIÓN

10.1 Requisitos Previos

Antes de instalar el proyecto, verifica que tengas:

- ✅ Python 3.10+ → Descargar
- ✅ Node.js 18+ → Descargar
- ✅ MySQL 8+ → Descargar
- ✅ Git → Descargar
- ✅ Visual Studio Code → Descargar
- ✅ MySQL Workbench → Descargar

10.2 Verificar Instalaciones

Abre una terminal (PowerShell o CMD) y ejecuta:

```
python --version
node --version
npm --version
git --version
```

Deben mostrar las versiones instaladas.

10.3 Clonar o Descargar el Proyecto

Opción A: Con Git

```
git clone https://github.com/TU_USUARIO/sistema-gestion-usuarios.git
cd sistema-gestion-usuarios
```

Opción B: Descargar ZIP

1. Descargar el ZIP del proyecto
2. Descomprimir en C:\Users\TuUsuario\sgu\

10.4 Configurar la Base de Datos

1. Abrir MySQL Workbench y conectar
2. Ejecutar el script sgu-backend/database.sql
3. Verificar:

```
USE gestion_usuarios_db;
SHOW TABLES;
```

Deben aparecer 6 tablas.

10.5 Configurar el Backend

```
# Entrar al backend
cd sgu-backend

# Crear entorno virtual
python -m venv venv

# Activar entorno virtual (Windows)
venv\Scripts\activate

# Instalar dependencias
pip install -r requirements.txt

# Configurar .env con tus datos de MySQL
# (ver sección 10.7)

# Ejecutar
python app.py
```

Si todo va bien verás:

```
============================================================
  SISTEMA DE GESTIÓN DE USUARIOS - BACKEND
  Servidor: http://127.0.0.1:5000
  CORS:     Aceptando peticiones de localhost:5173
============================================================
 * Running on http://127.0.0.1:5000
```

10.6 Configurar el Frontend

En otra terminal:

```
cd sgu-frontend
npm install
npm run dev
```

Verás:

```
VITE v8.3.2  ready in XXX ms
➜  Local:   http://localhost:5173/
```

10.7 Variables de Entorno (.env)

El archivo .env del backend debe contener:

```
SECRET_KEY=sgu_secret_key_2026
JWT_SECRET_KEY=sgu_jwt_secret_2026

DB_USER=root
DB_PASSWORD=tu_contraseña_mysql
DB_HOST=localhost
DB_PORT=3306
DB_NAME=gestion_usuarios_db
```

10.8 Acceder al Sistema

Abre el navegador en: http://localhost:5173/login

11. MANUAL DE USO

11.1 Iniciar Sesión

1. Abrir http://localhost:5173/login
2. Ingresar correo y contraseña
3. Clic en "Ingresar"

Credenciales de prueba:

Correo Contraseña nuevo_admin@sgu.com Nuevo123! juan@test.com Juan123!

11.2 Panel Principal (Dashboard)

Al iniciar sesión, verás el panel con:

· Resumen rápido: Contadores de usuarios activos e inactivos · Lista de usuarios: Todos los usuarios registrados · Buscador: Filtrar por nombre o correo · Filtros: Activos / Inactivos / Todos · Vista general / Actividad reciente: 2 pestañas

11.3 Crear Usuario

1. Clic en "Crear usuario"
2. Llenar: nombre, correo, contraseña, rol
3. Clic en "Guardar cambios"
4. El usuario aparecerá en la lista

11.4 Editar Usuario

1. Clic en el ícono ✏️ del usuario
2. Se abre un modal con los datos actuales
3. Modificar lo que se necesite
4. Clic en "Guardar cambios"

11.5 Desactivar Usuario (Eliminación Lógica)

1. Clic en el ícono 🗑️ del usuario
2. Confirmar en la ventana emergente
3. El usuario pasa a estado "Inactivo"
4. Ya no aparecerá en el filtro "Activos"

11.6 Reactivar Usuario

1. Cambiar al filtro "Inactivos"
2. Clic en el ícono 🔄 del usuario
3. Confirmar
4. El usuario vuelve a estar activo

11.7 Ver Auditoría

1. Clic en la pestaña "Actividad reciente"
2. Ver el historial de todas las acciones realizadas

3. Cada registro muestra: acción, usuario, fecha

11.8 Cerrar Sesión

1. Clic en "¿Cerrar sesión?" en el menú lateral
2. Confirmar en el modal
3. Vuelve a la pantalla de login

12. GUÍA DE PRUEBAS

12.1 Verificar que Todo Funciona

Prueba Resultado esperado

1 Abrir http://127.0.0.1:5000/api/health Responde JSON con "status: ok" 2 Abrir http://localhost:5173/login Carga la pantalla de login 3 Login con nuevo_admin@sgu.com / Nuevo123! Redirige al dashboard 4 Crear un usuario nuevo Aparece en la lista 5 Editar un usuario Los cambios se guardan 6 Desactivar un usuario Cambia a "Inactivo" 7 Reactivar el usuario Vuelve a "Activo" 8 Ver "Actividad reciente" Muestra el historial 9 Buscar por nombre Filtra la lista 10 Cerrar sesión Vuelve al login

12.2 Verificar la Base de Datos

Abre MySQL Workbench y ejecuta:

```
USE gestion_usuarios_db;
SELECT id_usuario, nombre, correo, estado FROM usuario;
```

Debe mostrar los usuarios registrados.

13. SOLUCIÓN DE PROBLEMAS

13.1 Error: "El puerto 5000 está en uso"

Algo más está usando el puerto. Soluciones:

· Reiniciar la PC · Cambiar el puerto en app.py: port=5001 · Cerrar procesos conflictivos

13.2 Error: "El puerto 5173 está en uso"

Vite usará automáticamente el 5174. Revisa la terminal para ver el puerto real.

13.3 Error: "No se puede activar el venv"

En PowerShell ejecuta:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Escribe S y Enter.

13.4 Error 401 Unauthorized

El token JWT expiró (dura 1 hora). Cerrar sesión y volver a iniciar.

13.5 Error de CORS

Verificar en app.py:

```
origins: ["http://localhost:5173"]
```

13.6 Error: "Access denied for user 'root'@'localhost'"

La contraseña del .env está mal. Corregir DB_PASSWORD.

13.7 Error: "Unknown database 'gestion_usuarios_db'"

La base de datos no existe. Ejecutar el database.sql en MySQL Workbench.

14. CONCLUSIONES

El desarrollo del Sistema de Gestión de Usuarios permitió consolidar los conocimientos adquiridos durante el programa de formación ADSO, aplicando de forma integral conceptos de:

· Análisis y diseño de software: Modelado de casos de uso, diagramas de clases, modelo entidad-relación y normalización. · Programación backend: Uso de Python con Flask, SQLAlchemy como ORM y aplicación de patrones de diseño. · Programación frontend: Framework Vue.js 3, enrutamiento con Vue Router, consumo de APIs con Axios. · Seguridad: Autenticación JWT, cifrado de contraseñas con bcrypt, control de acceso por roles y auditoría. · Bases de datos: MySQL relacional, normalización hasta 3FN, integridad referencial. · Control de versiones: Uso de Git y GitHub. · Documentación: Manuales de instalación, uso y pruebas.

El sistema resultante es una herramienta funcional y profesional que cumple con todos los requerimientos funcionales y no funcionales establecidos, y que podría adaptarse fácilmente a un entorno de producción real.

La experiencia adquirida en este proyecto es directamente aplicable al ámbito laboral, donde la gestión segura de usuarios es una necesidad constante en cualquier organización que maneje sistemas informáticos.

15. BIBLIOGRAFÍA

· Pressman, R. S. (2014). Ingeniería del software: un enfoque práctico (8ª ed.). McGraw-Hill. · Sommerville, I. (2011). Ingeniería de software (9ª ed.). Pearson. · Flask Documentation. (2026). Pallets Projects. https://flask.palletsprojects.com/ · Vue.js Documentation. (2026). Vue.js. https://vuejs.org/ · SQLAlchemy Documentation. (2026). SQLAlchemy. https://www.sqlalchemy.org/ · MySQL Documentation. (2026). Oracle Corporation. https://dev.mysql.com/doc/ · ISO/IEC 25010:2011. Systems and software Quality Requirements and Evaluation (SQuaRE). ISO.

FIN DEL DOCUMENTO

Sistema de Gestión de Usuarios (SGU) Juan Andrés Pérez Elguedo SENA - ADSO - 2026
